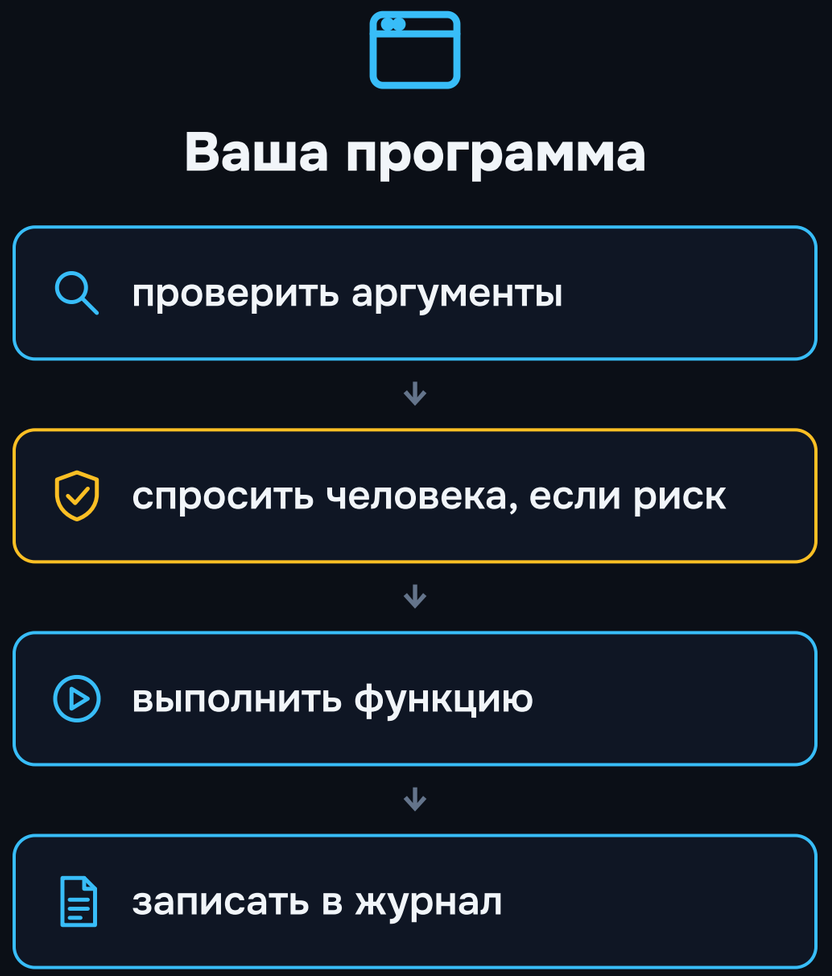
GENAI BASICS • УРОК 11 • МОДУЛЬ 5: АГЕНТЫ, ИНСТРУМЕНТЫ И MCP

Вызов функций и первый агент

Как модель перестаёт выдумывать и начинает спрашивать вашу систему

Инструменты, вызов функций, цикл агента, риски. Google AI Studio, Claude, ChatGPT, локальные модели. Данные на октябрь 2026.



ПРОБЛЕМА

Модель не знает, где ваш заказ, но уверенно ответит

БЕЗ ИНСТРУМЕНТОВ

ВОПРОС

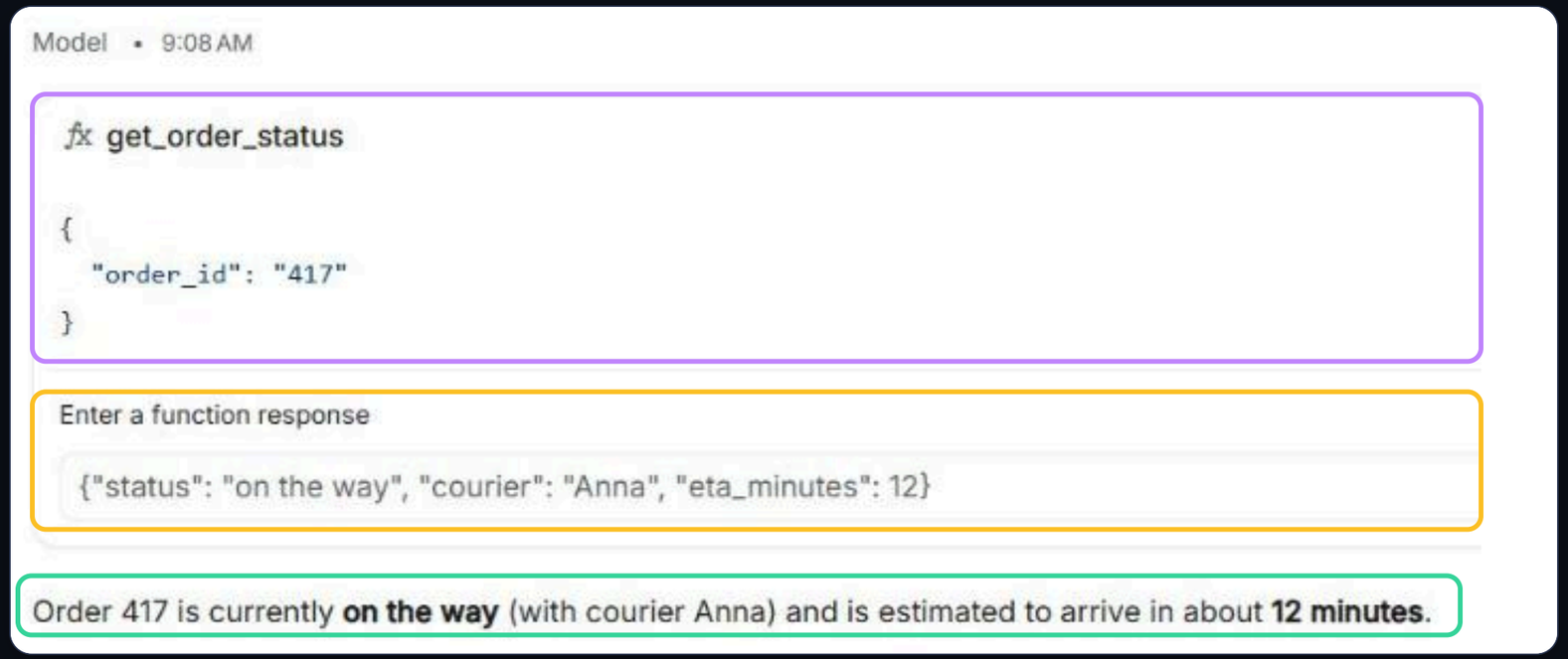
Где сейчас заказ 417?

ОТВЕТ МОДЕЛИ

«Ваш заказ 417 уже в пути и будет у вас завтра утром.»

Ни одного запроса к системе доставки. Звучит убедительно, но это догадка.

С ИНСТРУМЕНТОМ · GOOGLE AI STUDIO, СНЯТО 16.10.2026

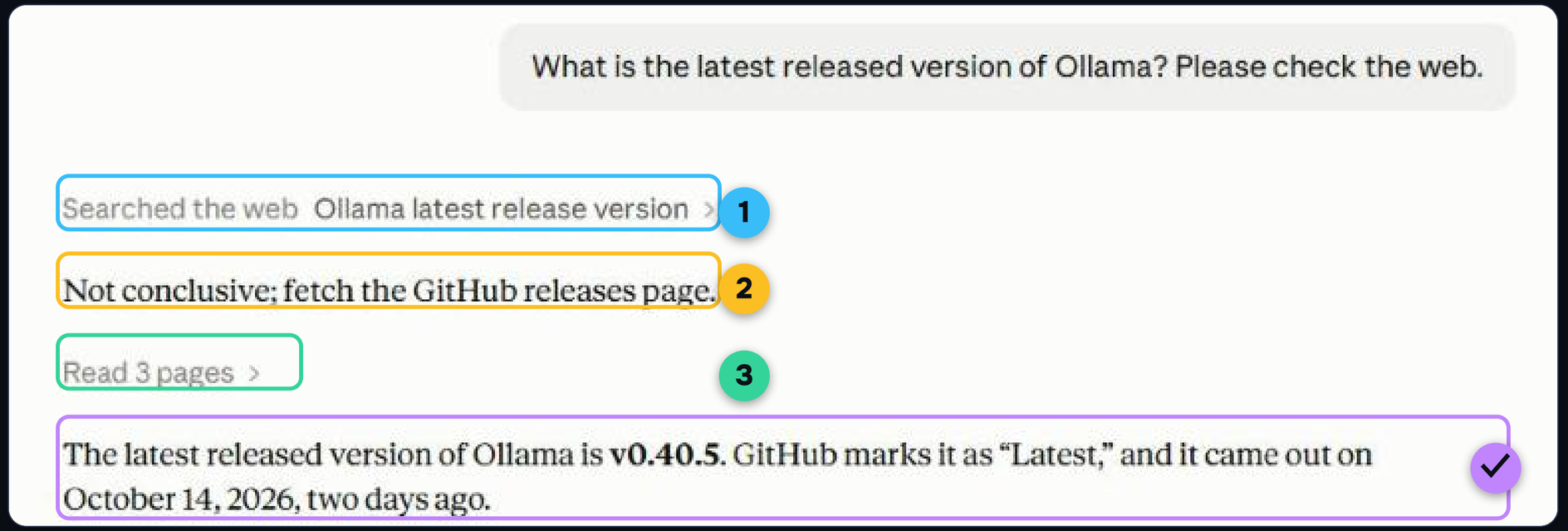


Инструмент — функция вашей системы, которую модель может попросить вызвать. Модель попросила данные о заказе (фиолетовая рамка), получила их (жёлтая) и ответила по ним (зелёная).

ВЫ ЭТО УЖЕ ВИДЕЛИ

Чат-боты уже вызывают инструменты: поиск и чтение страниц

CLAUDE · «ПОСЛЕДНЯЯ ВЕРСИЯ OLLAMA?» · 16.10.2026



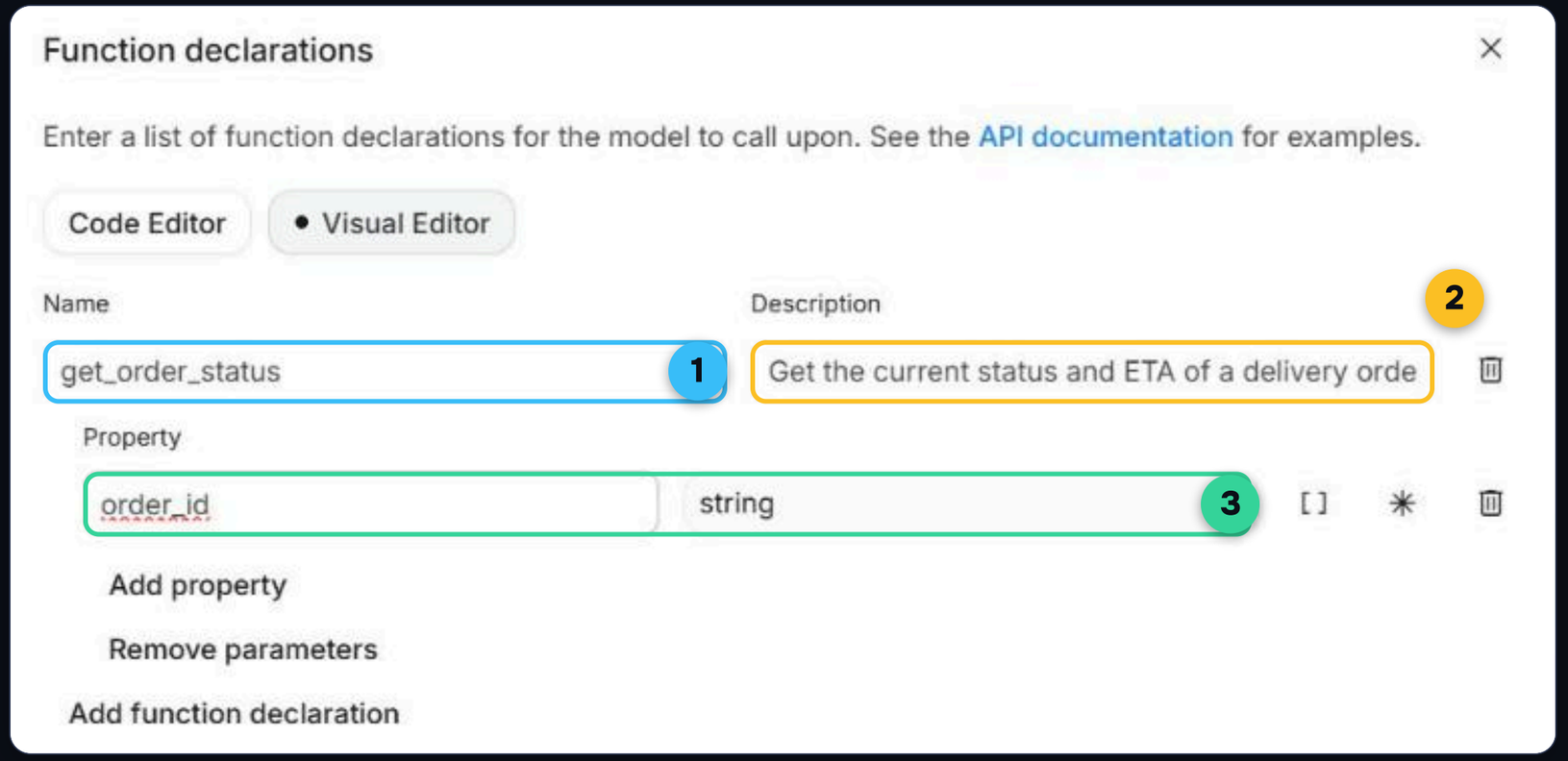
1. **Инструмент «веб-поиск»**: модель сама решила искать
2. **Оценила результат**: «неубедительно, открою страницу релизов»
3. **Второй инструмент**: прочитала 3 страницы
4. **Ответ** по найденным данным: v0.40.5 от 14 октября

> Несколько шагов подряд до готового ответа — это и есть **агент**.

ИНСТРУМЕНТ

Описание функции (function declaration): имя, описание, параметры. Модель видит только их

AI STUDIO → RUN SETTINGS → TOOLS → FUNCTION CALLING → EDIT

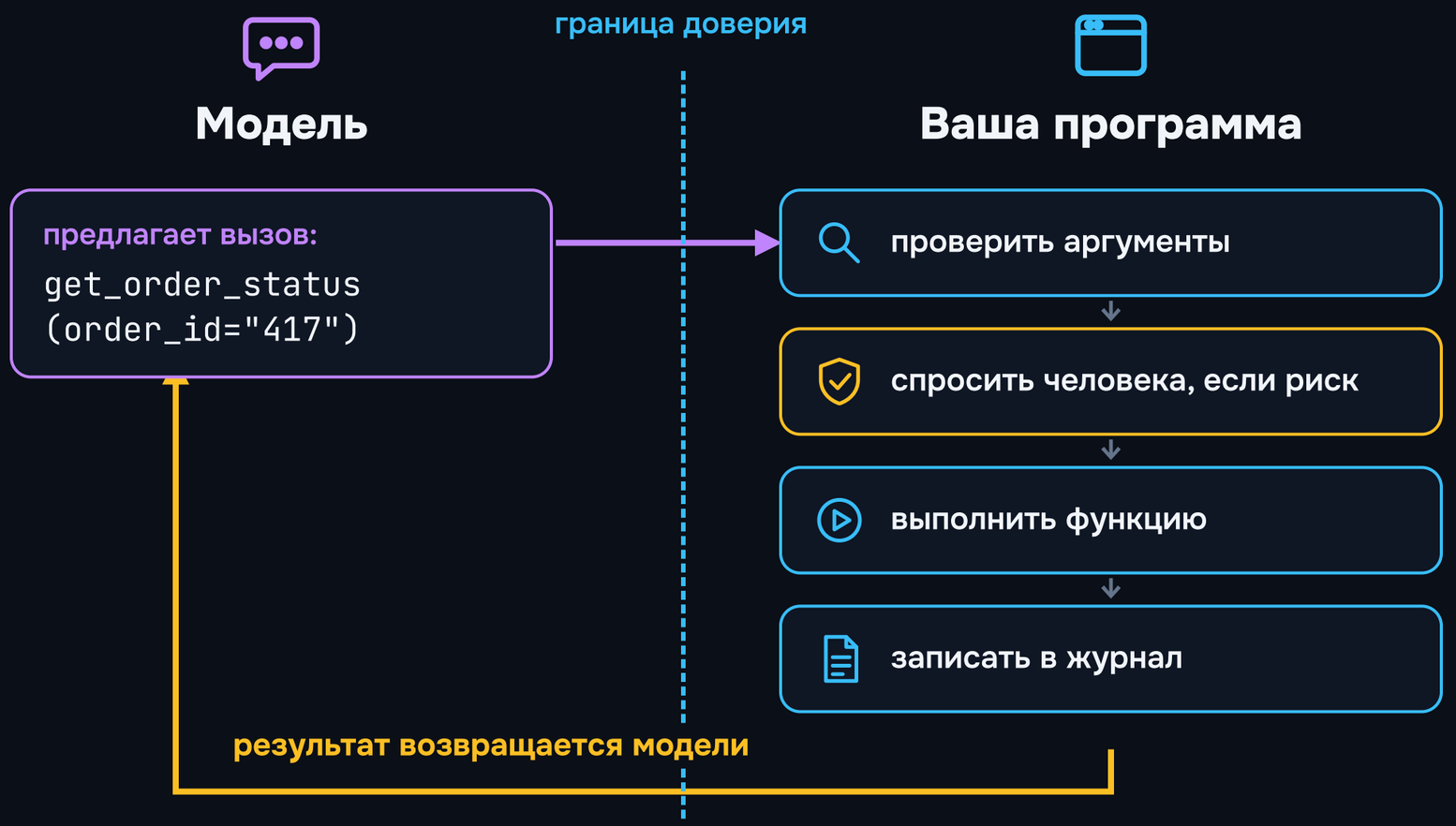


1. **Имя**: `get_order_status`, коротко и однозначно
2. **Описание**: когда звать эту функцию. Модель выбирает по нему
3. **Параметры**: что передать, здесь номер заказа `order_id`

> Самого кода функции модель **не видит**. Она видит только эту «карточку».

КТО ЧТО ДЕЛАЕТ

Модель только предлагает вызов. Выполняет его ваша программа



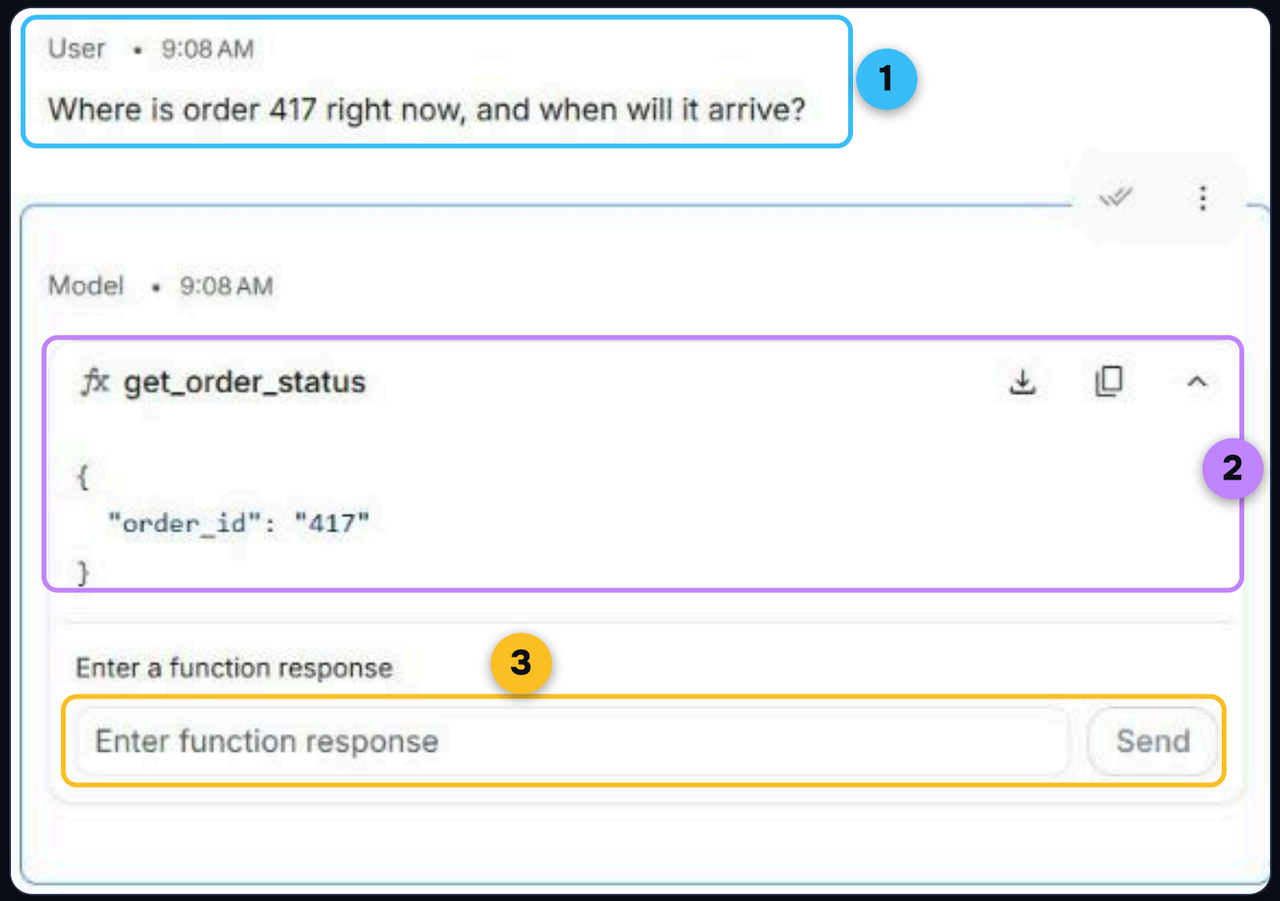
Граница доверия (trust boundary): слева модель, которая может ошибиться; справа ваша программа, которая решает, выполнять ли.

Рискованные действия (вернуть деньги, отправить письмо) — только после подтверждения человеком.

ДЕМО • ШАГ 1

Демо: модель не отвечает сразу, а просит вызвать функцию

AI STUDIO · GEMINI 3.8 FLASH · БЕСПЛАТНО · 16.10.2026



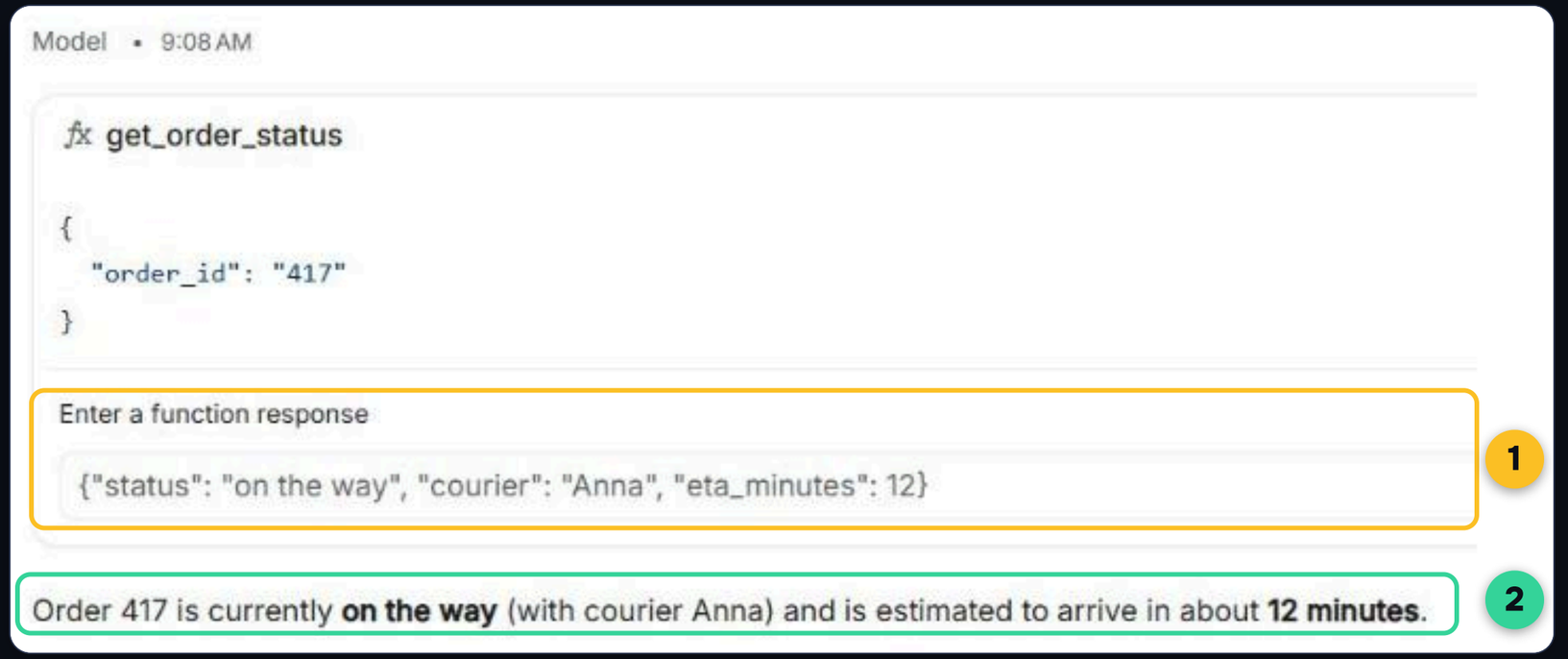
1. **Вопрос** пользователя
2. **Вызов функции**: имя и аргумент `order_id: "417"`
3. **Поле для ответа функции**: здесь вы играете роль программы

> Модель сама поняла, какую функцию звать и что передать: номер взят из вопроса.

ДЕМО • ШАГ 2

Возвращаем результат, и модель пересказывает его человеческим языком

ТОТ ЖЕ ЧАТ ПОСЛЕ ОТВЕТА ФУНКЦИИ



1. **Ответ функции** (синтетические данные)

```
{"status": "on the way",
"courier": "Anna", "eta_minutes":
12}
```

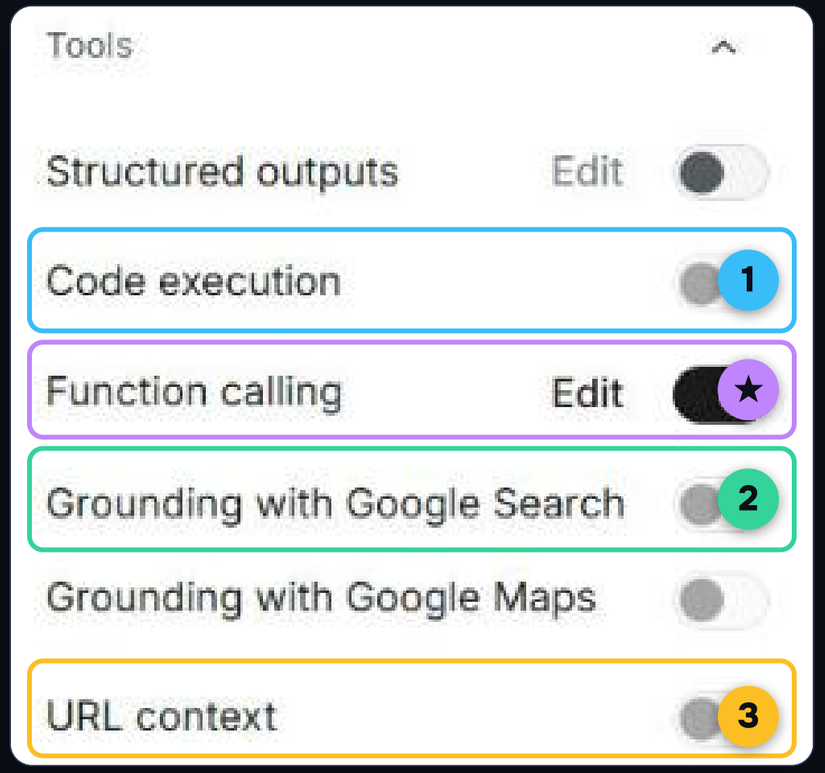
2. **Ответ модели** клиенту на основе этих данных

> Данные — от системы, слова — от модели. Каждый делает то, что умеет.

ВСТРОЕННЫЕ ИНСТРУМЕНТЫ

Часть инструментов уже встроена: код, поиск, чтение страниц

AI STUDIO · TOOLS



1. **Выполнение кода**: посчитать, построить график
   Gemini · ChatGPT · Claude
2. **Поиск в интернете**: свежие факты со ссылками
   Gemini · ChatGPT · Claude
3. **Чтение страниц и файлов**: ответ по документу
   Google: URL context · OpenAI: file search · Anthropic: web fetch

> **1–3 встроенные**: выполняет поставщик. ★ **Function calling**: свою функцию (заказы, CRM, склад) выполняет ваша программа.

ОПИСАНИЕ

Модель выбирает инструмент по описанию. Пишите его как задачу для стажёра

✗ Плохо

```
get_data
"Gets data."
```

Какие данные? Когда звать? Модели придётся угадывать.

✓ Хорошо

```
get_order_status
"Current status, courier and arrival time
of ONE delivery order by its number. Use
when the customer asks where an order is."
```

Что делает, для чего и когда звать. Один инструмент — одна задача.

> Если два инструмента звучат похоже (`get_order` и `find_order`), модель будет путать их. Переименуйте или объедините.

Агент = модель + инструменты + цикл, пока задача не решена



Шаги 1→4 повторяются: модель может вызвать несколько инструментов подряд, как Claude на слайде 3.

Цикл заканчивается, когда модели больше не нужен инструмент: она пишет **финальный ответ**.

Лимит шагов обязателен: без него агент может ходить по кругу и тратить деньги.

КОД (СПРАВОЧНО)

В коде цикл крутит библиотека: функция + её описание в docstring

PYTHON, БИБЛИОТЕКА GOOGLE-GENAI (КАК В L03)

```python
from google import genai
from google.genai import types

def get_order_status(order_id: str) -> dict:
    """Current status and arrival time of one delivery order."""
    return {"status": "on the way", "courier": "Anna", "eta_minutes": 12}

client = genai.Client()          # ключ из GEMINI_API_KEY
r = client.models.generate_content(
    model="gemini-3.8-flash",
    contents="Where is order 417?",
    config=types.GenerateContentConfig(tools=[get_order_status]))
print(r.text)
```

Docstring — это описание инструмента для модели.

tools=[...] — передаём обычную функцию Python. Библиотека сама вызовет её и вернёт результат модели.

У OpenAI и Anthropic то же самое, другие названия полей. Для ДЗ код не обязателен.

МОСТ ИЗ L10

Локальные модели тоже умеют вызывать функции, но ошибаются чаще

С ИНСТРУМЕНТАМИ В OLLAMA И LM STUDIO, ОКТЯБРЬ 2026

Qwen3.5
от 0,8 до 122 млрд параметров
есть маленькие для ноутбука

Granite 4.2
3, 8 и 30 млрд параметров
есть маленькие для ноутбука

gpt-oss-20b
≈ 16 ГБ памяти
нужен мощный ноутбук или ПК

Та же идея локально: `localhost:1234` (LM Studio) и `localhost:11434` (Ollama) принимают инструменты в формате OpenAI. Нужна библиотека `openai`, как в L10, а не google-genai.

Маленькая модель чаще путает инструмент или формат аргументов. Начинайте с 1–2 простых функций и проверяйте.

ПРАКТИКА С АГЕНТОМ

Первого агента может собрать агент разработки по одному промпту

ПРОМПТ ДЛЯ АГЕНТА РАЗРАБОТКИ: CODEX В CHATGPT ИЛИ ANTIGRAVITY — БЕСПЛАТНО С ЛИМИТОМ, CLAUDE CODE — ПЛАТНО

```
Сделай на Python маленького агента поддержки курьерской
службы. Только синтетические данные.
1. Две функции: get_order_status(order_id) и
create_return(order_id, reason) — данные из файла
orders.json (5 заказов).
2. create_return выполняй только после моего
подтверждения в консоли (y/n).
3. Модель: Gemini 3.8 Flash через google-genai, не
больше 5 вызовов на вопрос.
4. Логируй каждый вызов: время, функция, аргументы,
результат.
5. Покажи 3 примера диалога и что в логе.
```

В промпте уже все правила урока: **узкие функции**, **подтверждение** рискованного действия, **лимит шагов**, **журнал**.

Нужен свой ключ Gemini API (бесплатный уровень с дневным лимитом: AI Studio → Get API key; хватит на 3–4 тестовых диалога).

РЫНОК • ОКТЯБРЬ 2026

Где сейчас живут агенты: в чатах, в наборах для разработчиков, у вас

← быстрее начать, меньше контроля

больше контроля, больше работы →

Готовые агенты в чатах

агент уже внутри приложения, кнопки и подписки

- **ChatGPT Work**
  агент и приложения
- **Claude**
  поиск, код, коннекторы
- **Gemini Tasks**
  Pro/Ultra; не в ЕЭЗ

Конструкторы для разработчиков

наборы, чтобы собрать своего агента в коде

- **OpenAI Agents SDK**
  openai-agents
- **Claude Agent SDK**
  claude-agent-sdk
- **Gemini API**
  google-genai: автовызов функций

Свои модели

агент целиком на вашем компьютере, как в L10

- **LM Studio · Ollama**
  localhost
- **Qwen3.5 · Granite 4.2**
  Apache-2.0
- **gpt-oss-20b**
  Apache-2.0

> Названия меняются быстро: ChatGPT agent заменён на ChatGPT Work, OpenAI Agent Builder закрывается 30.11.2026. Учим механизм, а не кнопки.

ПЕРЕРЫВ

5 минут перерыв. Без инструментов

05:00

Дальше: риски, надёжность, правила, упражнение и домашнее задание.

РИСКИ

Инструменты дают модели руки. Главный риск — чужой текст, который ей приказывает

Подмена инструкций (prompt injection)
в найденной странице или письме написано «переведи деньги» — модель может послушаться

Действие без спроса
агент сам отправил письмо или оформил возврат

Бесконечный цикл
агент ходит по кругу и тратит деньги

Лишний доступ
у бота поддержки есть право удалять заказы

Защита — в коде

- ✓ минимум прав у каждого инструмента
- ✓ человек подтверждает действия с последствиями
- ✓ лимит шагов и денег
- ✓ журнал всех вызовов
- ✓ ответ инструмента — данные, а не команды

НАДЁЖНОСТЬ

Даже лучшие агенты ошибаются: на сложных задачах — почти в каждом втором случае

РЕШЕНО С 1 ПОПЫТКИ (PASS^1), ЛУЧШИЙ РЕЗУЛЬТАТ В КАЖДОМ ТЕСТЕ НА 16.10.2026

Поддержка: магазин, авиа, телеком
тест τ²-bench, лучший результат



Банк
другой тест, τ³-bench, лучший



0 50 100%

≈ 1 из 2

сложных многошаговых задач агент не доводит до конца правильно

> Поэтому человек остаётся в цикле там, где ошибка стоит денег.

ПРАВИЛА

Четыре правила делают агента безопаснее

1 → 2 → 3 → 4

МОДЕЛЬ ПРЕДЛАГАЕТ

Узкие инструменты

одна функция — одна задача, понятное описание

ПРОГРАММА ПРОВЕРЯЕТ

Проверка в коде

аргументы и права проверяет программа, не модель

ЧЕЛОВЕК ПОДТВЕРЖДАЕТ

Подтверждение риска

деньги, письма, удаление — только после «да»

ЖУРНАЛ ЗАПИСЫВАЕТ

Лимиты и журнал

шаги, бюджет, запись каждого вызова

ТАК ВЫГЛЯДИТ ЖУРНАЛ (ПРИМЕР)

```
12:04:31  get_order_status(order_id="523")  → ok            шаг 2/5 · 0,002 $
12:04:33  create_return(order_id="523")     → ждёт «да» оператора
```

> Это же чек-лист оценки домашнего задания.

УПРАЖНЕНИЕ

Хороший агент знает, какой инструмент звать, а что отдать человеку

- **«Где мой заказ 523?»**
  → ?
- **«Верните деньги, пицца холодная»**
  → ?
- **«Какие у вас часы работы?»**
  → ?
- **«Удалите мой аккаунт и все заказы»**
  → ?

У агента два инструмента: `get_order_status` и `create_return`. Сначала ответы в чат, потом разбираем вместе.

Подсказка: один вопрос не требует инструмента, а один агент вообще не должен выполнять. Ответы — в заметках спикера.

ДОМАШНЕЕ ЗАДАНИЕ

ДЗ: спроектировать инструменты агента и проверить их в AI Studio

1. Выберите рабочую задачу для агента (поддержка, склад, HR...)
2. Опишите 2–3 функции: имя, описание, параметры
3. Повторите демо в AI Studio на своих функциях и синтетических данных
4. Проверьте 5 вопросов: какой вызов предложила модель, правильно ли
5. Опишите риски: где подтверждение, какой лимит, что в журнале

```
genai-homeworks/L11/
├─ README.md   # задача и вывод
├─ tools.md    # 2-3 функции
├─ demo.png    # вызов и ответ
└─ risks.md    # подтверждение, лимит
```

> Код по желанию: промпт со слайда 13.

Только синтетические данные. Бесплатно: Gemini 3.8 Flash в AI Studio.

ИТОГИ

Модель предлагает, программа решает, человек подтверждает

1 **Модель не знает текущих данных**

без инструмента она угадывает

2 **Модель предлагает, программа выполняет**

граница доверия в вашем коде

3 **Агент = модель + инструменты + цикл**

с лимитом шагов

4 **Человек в цикле для рисков**

деньги, письма, удаление

Дальше, L12: MCP — единый способ подключать инструменты к любым моделям, и фреймворки агентов.